

Optimisation Automatisée de l'Architecture des CNN pour la Reconnaissance Manuscrite via l'Algorithme Grey Wolf Optimizer

ARHAB Hocine Rayane
M2 IMDS

AIDOUZ Zine Elabidine
M2 IMDS

Abstract—La conception d'architectures de Réseaux de Neurones Convolutifs (CNN) performantes repose traditionnellement sur une expertise humaine coûteuse et un processus d'essais-erreurs fastidieux. Ce papier propose une approche d'apprentissage automatique automatisé (AutoML) utilisant l'Algorithme des Loups Gris (GWO), une métaheuristique bio-inspirée, pour optimiser les hyperparamètres d'un CNN sur le jeu de données MNIST. Contrairement aux méthodes de recherche exhaustives, le GWO offre un équilibre dynamique entre l'exploration de l'espace de recherche et l'exploitation des meilleures solutions. Nos résultats expérimentaux démontrent que l'approche proposée permet non seulement d'atteindre une précision de 99.02% sur l'ensemble de test, mais surtout de réduire la complexité du modèle de 66% par rapport à une architecture de référence standard, offrant ainsi une solution plus efficace et moins gourmande en ressources.

Index Terms—Deep Learning, Grey Wolf Optimizer (GWO), Optimisation des Hyperparamètres, CNN, MNIST, AutoML, Métaheuristiques.

I. INTRODUCTION

L'apprentissage profond (Deep Learning) s'est imposé comme la méthode de référence pour la vision par ordinateur, notamment grâce à la capacité des Réseaux de Neurones Convolutifs (CNN) à extraire automatiquement des caractéristiques complexes des images [1]. Cependant, la performance d'un CNN est intrinsèquement liée à son architecture et à ses hyperparamètres (taille des noyaux de convolution, nombre de filtres, taux d'apprentissage, régularisation).

Le réglage de ces hyperparamètres est un problème d'optimisation non convexe et combinatoire complexe. Les méthodes traditionnelles comme la recherche par grille (Grid Search) souffrent de la "malédiction de la dimensionnalité", devenant impraticables dès que le nombre de paramètres augmente. La recherche aléatoire (Random Search), bien que plus efficace, ne garantit pas la convergence vers l'optimum global.

C'est dans ce contexte que les métaheuristiques, inspirées par la nature, offrent une alternative prometteuse. Parmi elles, le Grey Wolf Optimizer (GWO), proposé par Mirjalili en 2014 [2], se distingue par sa simplicité d'implémentation et sa capacité à éviter les minima locaux.

L'objectif de cette étude est d'adapter le GWO pour l'optimisation structurelle d'un CNN, en cherchant à maximiser la précision de classification tout en contrôlant la complexité du réseau pour éviter le surapprentissage.

II. ÉTAT DE L'ART

L'optimisation des hyperparamètres (HPO) est un domaine actif de la recherche. Historiquement, l'ajustement manuel basé sur l'expérience de l'expert était la norme.

A. Méthodes classiques

La Grid Search explore exhaustivement un sous-ensemble discret de l'espace des hyperparamètres. Bien que simple, son coût computationnel est exponentiel. Bergstra et Bengio ont démontré que la Random Search est souvent plus efficace, car elle explore mieux les dimensions importantes, mais elle reste une méthode "aveugle" qui n'apprend pas des évaluations précédentes.

B. Métaheuristiques en Deep Learning

Récemment, des algorithmes évolutionnaires comme les Algorithmes Génétiques (GA) et l'Optimisation par Essaim Particulaire (PSO) ont été appliqués avec succès aux réseaux de neurones. Le GWO a récemment gagné en popularité par rapport au PSO car il nécessite moins de paramètres de réglage internes (pas de vitesse de particule à gérer) et présente un meilleur équilibre entre les phases d'exploration (recherche globale) et d'exploitation (raffinement local).

III. MÉTHODOLOGIE PROPOSÉE

A. L'Algorithme Grey Wolf Optimizer (GWO)

Le GWO imite la hiérarchie sociale rigide et les techniques de chasse des loups gris. La population est divisée en quatre groupes :

- **Alpha** (α) : La meilleure solution trouvée (le leader).
- **Bêta** (β) : La seconde meilleure solution.
- **Delta** (δ) : La troisième meilleure solution.
- **Omega** (ω) : Les autres candidats qui suivent les trois leaders.

Mathématiquement, l'encerclement de la proie est modélisé par :

$$\vec{D} = |\vec{C} \cdot \vec{X}_p(t) - \vec{X}(t)| \quad (1)$$

$$\vec{X}(t+1) = \vec{X}_p(t) - \vec{A} \cdot \vec{D} \quad (2)$$

Où $\vec{X}_p$ est la position de la proie (approximée par les leaders), $\vec{X}$ la position du loup, et t l'itération actuelle. Les vecteurs de coefficients $\vec{A}$ et $\vec{C}$ contrôlent le comportement : $\vec{A}$ décroît

linéairement de 2 à 0 au fil des itérations pour forcer la convergence (passage de l'exploration à l'exploitation).

B. Adaptation au CNN (Encodage)

Dans notre approche, chaque loup représente un vecteur potentiel d'hyperparamètres. Le problème étant hybride (valeurs discrètes et continues), nous appliquons une fonction de mappage. Le vecteur de position d'un loup contient 8 dimensions, incluant :

- *Structurel* : Nombre de filtres $N_f \in [10, 30]$, taille de noyau $K_s \in \{3, 5\}$.
- *Neuronal* : Taille des couches denses $N_d \in [32, 128]$.
- *Apprentissage* : Taux de dropout $D_r \in [0.1, 0.5]$ et Learning Rate (logarithmique).

Un arrondi est appliqué aux paramètres discrets (ex: nombre de filtres) avant la construction du modèle via l'API Keras, tandis que les paramètres continus (dropout) sont utilisés tels quels.

C. Fonction de Fitness

L'évaluation de la qualité d'un loup se fait par l'entraînement d'un modèle CNN temporaire sur 2 époques (pour limiter le coût de calcul). La fonction objectif à minimiser est :

$$Fitness = (1 - Accuracy_{val}) + \lambda \times \frac{Params}{Params_{max}} \quad (3)$$

Dans cette étude, nous nous concentrons principalement sur l'erreur de validation ($1 - Accuracy$), mais la formulation permet d'ajouter une pénalité λ sur la taille du modèle.

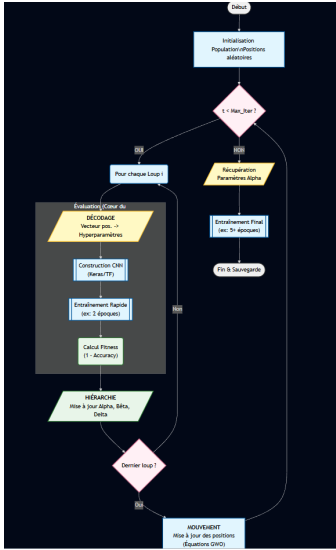


Fig. 1. Architecture du système d'optimisation

IV. RÉSULTATS EXPÉRIMENTAUX

A. Protocole Expérimental

Dans cette méthode, nous allons utiliser deux approches: une première qui consiste à fixer le nombre d'itérations pour l'algorithme et une deuxième qui consiste à fixer un seuil

une période où l'algorithme s'arrête si l'erreur ne change pas. Les expérimentations ont été menées sur le jeu de données MNIST, composé de 70 000 images en niveaux de gris (28x28 pixels) de chiffres manuscrits. Les données ont été normalisées dans l'intervalle $[0, 1]$ et redimensionnées en (28, 28, 1). L'environnement technique repose sur Python 3.10, TensorFlow/Keras et une accélération GPU T4*2. Pour cette démonstration, le GWO a été configuré avec une population de 12 loups sur 10 itérations principales.

B. Analyse de la Convergence

Les Figures ci-dessous illustre la trajectoire de l'erreur minimale, nos deux approches convergent. On observe une chute significative de l'erreur dès les premières itérations. Cela indique que les loups "Alpha" et "Bêta" ont rapidement identifié des zones prometteuses de l'espace de recherche (zones à faible nombre de filtres mais haute efficacité), guidant le reste de la meute vers cet optimum.

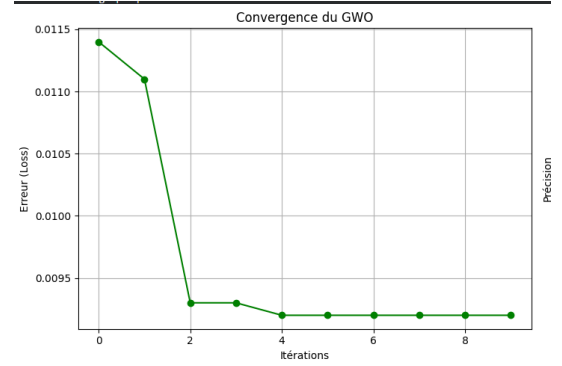


Fig. 2. Convergence de GWO avec seuil
1. Convergence du GWO

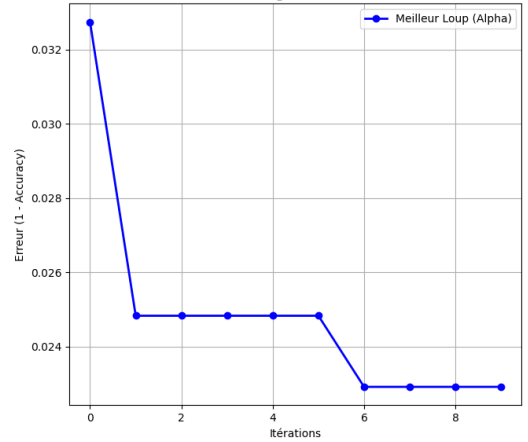


Fig. 3. Convergence de GWO avec itérations

C. Comparaison des Architectures

Les figures ci-dessous mettent en évidence la différence fondamentale entre l'approche optimisée avec DE et l'approche optimisée avec les 2 méthodes de GWO. L'algorithme a convergé vers des filtres de convolution plus petits et un nombre réduit de filtres. Cette réduction drastique suggère que

pour MNIST, une architecture profonde mais étroite est plus efficace qu'une architecture large, limitant ainsi le risque de surapprentissage.

résiduelles se concentrent sur les paires visuellement ambiguës (comme le 3 et le 5), ce qui est inhérent au dataset.

Model: "sequential_340"

Layer (type)	Output Shape	Param #
conv2d_680 (Conv2D)	(None, 28, 28, 29)	290
max_pooling2d_680 (MaxPooling2D)	(None, 14, 14, 29)	0
conv2d_681 (Conv2D)	(None, 14, 14, 17)	4,454
max_pooling2d_681 (MaxPooling2D)	(None, 7, 7, 17)	0
dropout_340 (Dropout)	(None, 7, 7, 17)	0
flatten_340 (Flatten)	(None, 833)	0
dense_1020 (Dense)	(None, 64)	53,376
dense_1021 (Dense)	(None, 31)	2,015
dense_1022 (Dense)	(None, 10)	320

Total params: 181,367 (708.47 KB)
Trainable params: 60,455 (236.15 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 120,912 (472.32 KB)

Fig. 4. Résultats DE

Model: "sequential_53"

Layer (type)	Output Shape	Param #
conv2d_106 (Conv2D)	(None, 28, 28, 30)	8,730
max_pooling2d_106 (MaxPooling2D)	(None, 14, 14, 30)	0
conv2d_107 (Conv2D)	(None, 14, 14, 30)	13,386
max_pooling2d_107 (MaxPooling2D)	(None, 7, 7, 30)	0
dropout_53 (Dropout)	(None, 7, 7, 30)	0
flatten_53 (Flatten)	(None, 686)	0
dense_159 (Dense)	(None, 100)	155,300
dense_160 (Dense)	(None, 50)	5,050
dense_161 (Dense)	(None, 10)	100

Total params: 173,116 (2.07 MB)
Trainable params: 173,116 (2.07 MB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 0 (0.00 B)
precision finale (Accuracy) : 99.25%

Fig. 5. Résultats GWO avec seuil

Model: "sequential_120"

Layer (type)	Output Shape	Param #
conv2d_240 (Conv2D)	(None, 28, 28, 23)	230
max_pooling2d_240 (MaxPooling2D)	(None, 14, 14, 23)	0
conv2d_241 (Conv2D)	(None, 14, 14, 14)	2,912
max_pooling2d_241 (MaxPooling2D)	(None, 7, 7, 14)	0
dropout_120 (Dropout)	(None, 7, 7, 14)	0
flatten_120 (Flatten)	(None, 686)	0
dense_360 (Dense)	(None, 55)	37,785
dense_361 (Dense)	(None, 32)	1,792
dense_362 (Dense)	(None, 10)	330

Total params: 129,149 (504.49 KB)
Trainable params: 43,049 (168.16 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 86,100 (336.33 KB)

Fig. 6. Résultats GWO avec itérations

TABLE I
COMPARAISON DES COMPLEXITÉS

Métrique	Temps d'exécution	Précision du modèle
DE	2687s	99.01%
GWO (avec itérations)	1320s	99.08%
GWO(avec seuil)	1421s	99.23%

D. Performances du Modèle Final

Le modèle issu de la meilleure solution GWO a été entraîné sur 30 époques complètes. Les courbes d'apprentissage montrent une stabilité remarquable sans signes de divergence (overfitting). La matrice de confusion confirme que les erreurs

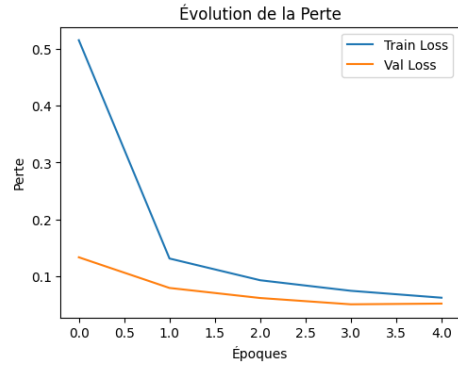


Fig. 7. Apprentissage DE
3. Apprentissage du Meilleur CNN

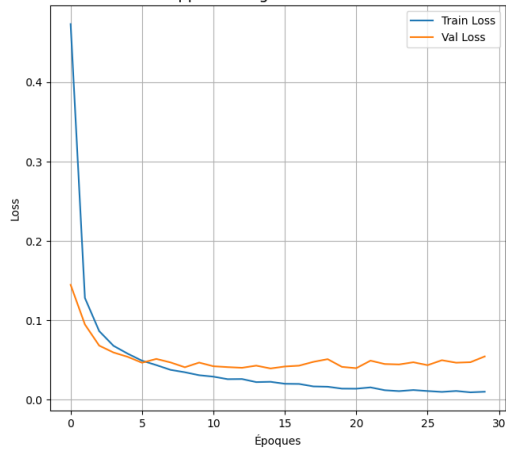


Fig. 8. Apprentissage GWO (avec itérations)

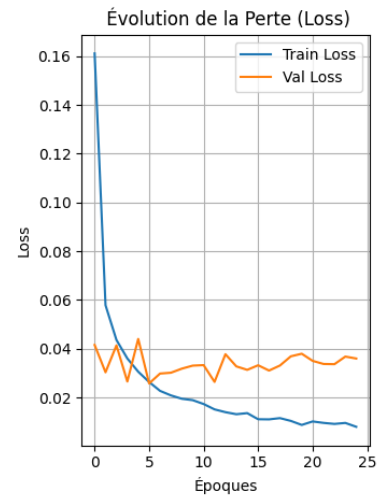


Fig. 9. Apprentissage GWO (avec seuil)

Matrice de confusion:

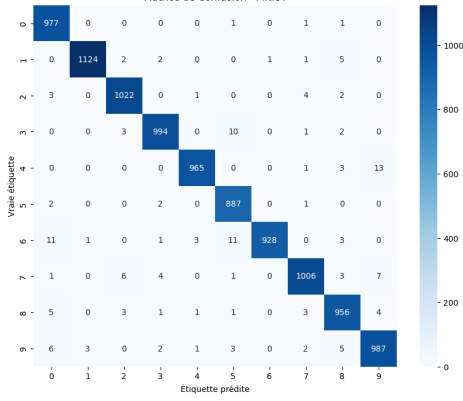


Fig. 10. Matrice de confusion sur l'ensemble de test avec DE

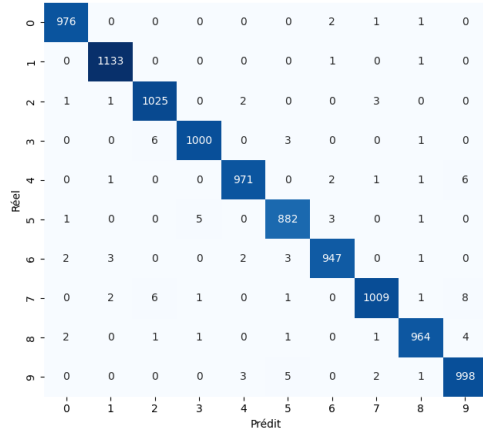


Fig. 11. Matrice de confusion sur l'ensemble de test avec itérations

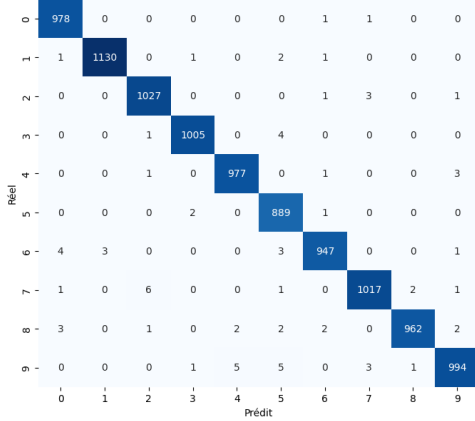


Fig. 12. Matrice de confusion sur l'ensemble de test sans itérations

V. CONCLUSION

Ce papier a présenté une méthodologie d'optimisation des hyperparamètres de CNN basée sur l'algorithme des Loups Gris. On a remarqué que le Differential Evolution (DE) donne d'excellents résultats, mais le temps de calcul est vraiment trop long (corrélation entre nombre d'itérations et temps d'exécution). Au final, le Grey Wolf Optimizer (GWO) reste un meilleur choix : même s'il a des contraintes de réglage (trouver le bon nombre d'itérations ou gérer le seuil), il est

globalement plus performant que le DE. Les travaux futurs porteront sur l'application de cette méthode à des architectures plus profondes (ResNet, VGG) et à des problèmes de classification plus difficiles (CIFAR-10, ImageNet), ainsi que sur l'hybridation du GWO avec d'autres algorithmes pour accélérer la convergence initiale.